

为什么大模型数学 RL 没有像 AlphaZero 那样轻易解决？

汇报人：杨凯森，黄禹

猜测一：是不是因为状态 / 动作空间太大？

这个猜测很自然

围棋只有 361 个格子，而大模型词表可以有 16 万 token。

是不是因为搜索空间太大，所以 RL 很难像 AlphaZero 那样迭代？

但动作集合可以工程化收缩

模型词表可以很大，但数学推理真正需要活跃保留的符号集合小得多。

如果只考虑数学符号和推理格式，几百个 token 量级就能覆盖相当多场景。

160k tokens -> ~300 math tokens

猜测二：是不是因为合法性约束太弱？

这个猜测也很自然

围棋每一步几乎都是合法落子。

而语言模型会写出很多非法表达、无意义推导、格式错误答案。

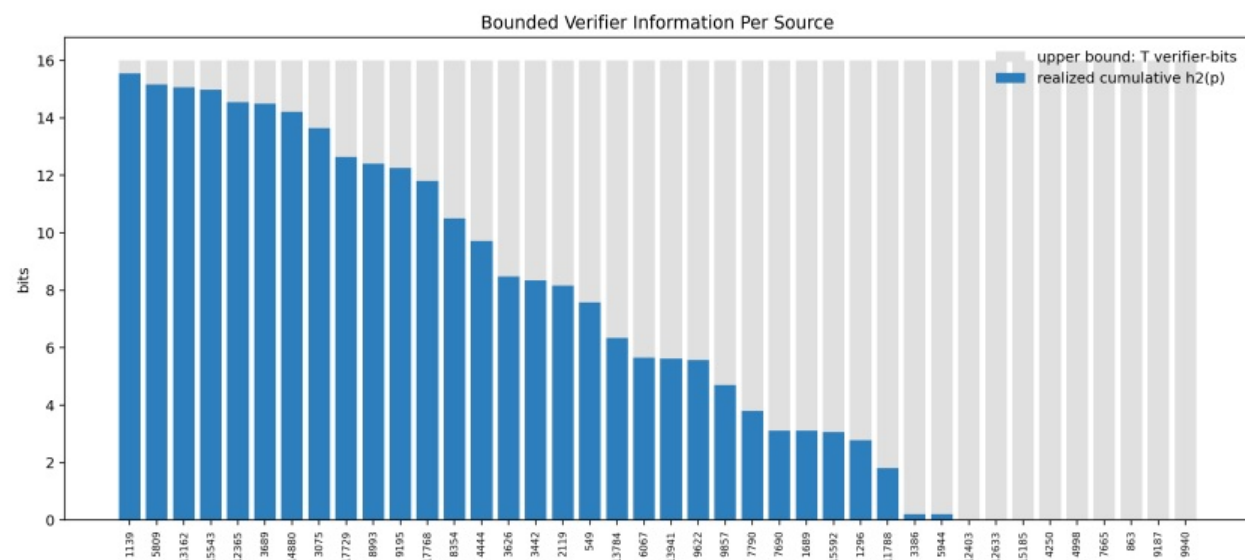
但合法性可以交给编译器

可以接入 Lean / parser / 类型检查器，在生成过程中做静态检查。

这样模型不一定能写出正确证明，但至少可以被限制在合法表达附近。

LM + compiler feedback

学术界的大模型数学 RL，多在固定大小数据集上实验



固定题集意味着固定信息源

给定训练集 $D = \{q_1, \dots, q_n\}$, RL 反复在这些题目上 rollout, 并用 verifier reward 更新模型。

如果每道题 q_i 的信息积分有上限 C_i , 那么整个固定数据集的信息量也有上限。

$$A_D(T) = \sum_i A_i(T) \leq \sum_i C_i$$

左图展示的是每个 source prompt 已实现的信息积分与理论上限。

我们怎么定义 RL 的平均信息量？

给定题目和 rollout

题目 q_i , 训练 step t 的模型是 π_t 。
一次 rollout 是从当前模型采样一条解答轨迹。

$$\tau_i(t) \sim \pi_t(. \mid q_i)$$

verifier 给出 reward。

$$R_i(t) = r(q_i, \tau_i(t))$$

一次 rollout 的平均信息量

先把 reward 仿射变换到 0-1, 避免不同 reward 尺度不可比。

$$R01_i(t) = (R_i(t) - R_{\min}) / (R_{\max} - R_{\min})$$

一次 rollout 属于当前 reward 分布; 该分布的方差就是它对应的平均信息量。

$$I_i(t) = V[R01_i(t)]$$

二值 reward 时:

$$I_i(t) = p_i(t)(1 - p_i(t)) \leq 1/4$$

信息积分

沿训练步数累加, 就得到题目 q_i 的信息积分。

$$A_i(T) = \sum_{t=0}^{T-1} I_i(t)$$

固定数据集 D 的总信息积分:

$$A_D(T) = \sum_{q_i \in D} A_i(T)$$

如果 $p_i(t)$ 接近 0 或 1, rollout reward 几乎不再有差异, $I_i(t)$ 接近 0; 只有 $p_i(t)$ 接近 1/2 时, 每次 rollout 的平均信息量最大。

我们先看 AlphaZero：它为什么不会很快耗尽信息？

背景：自博弈 + MCTS

当前 policy/value network 通过 MCTS 搜索来落子。

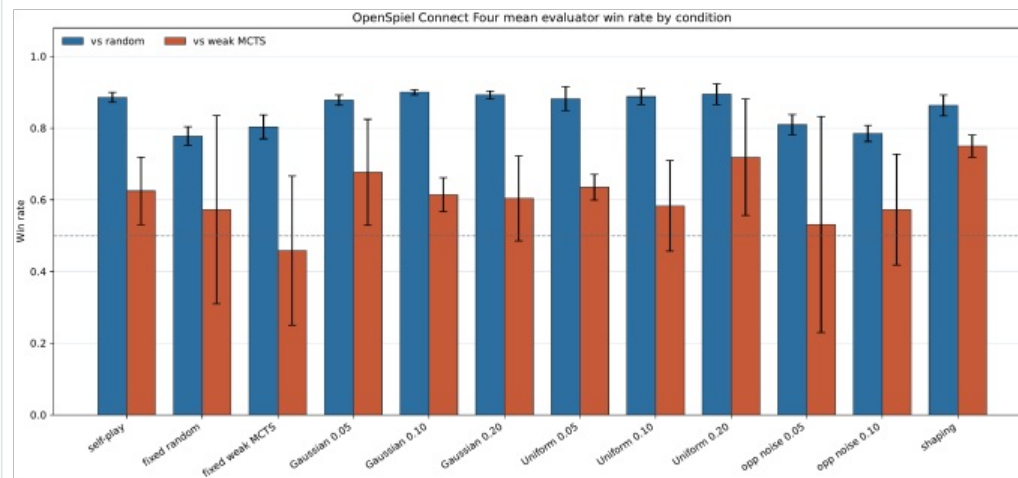
模型和自己对弈，胜负结果再训练新的 policy/value network。

新模型继续成为下一轮自己的对手，数据分布会被不断刷新。

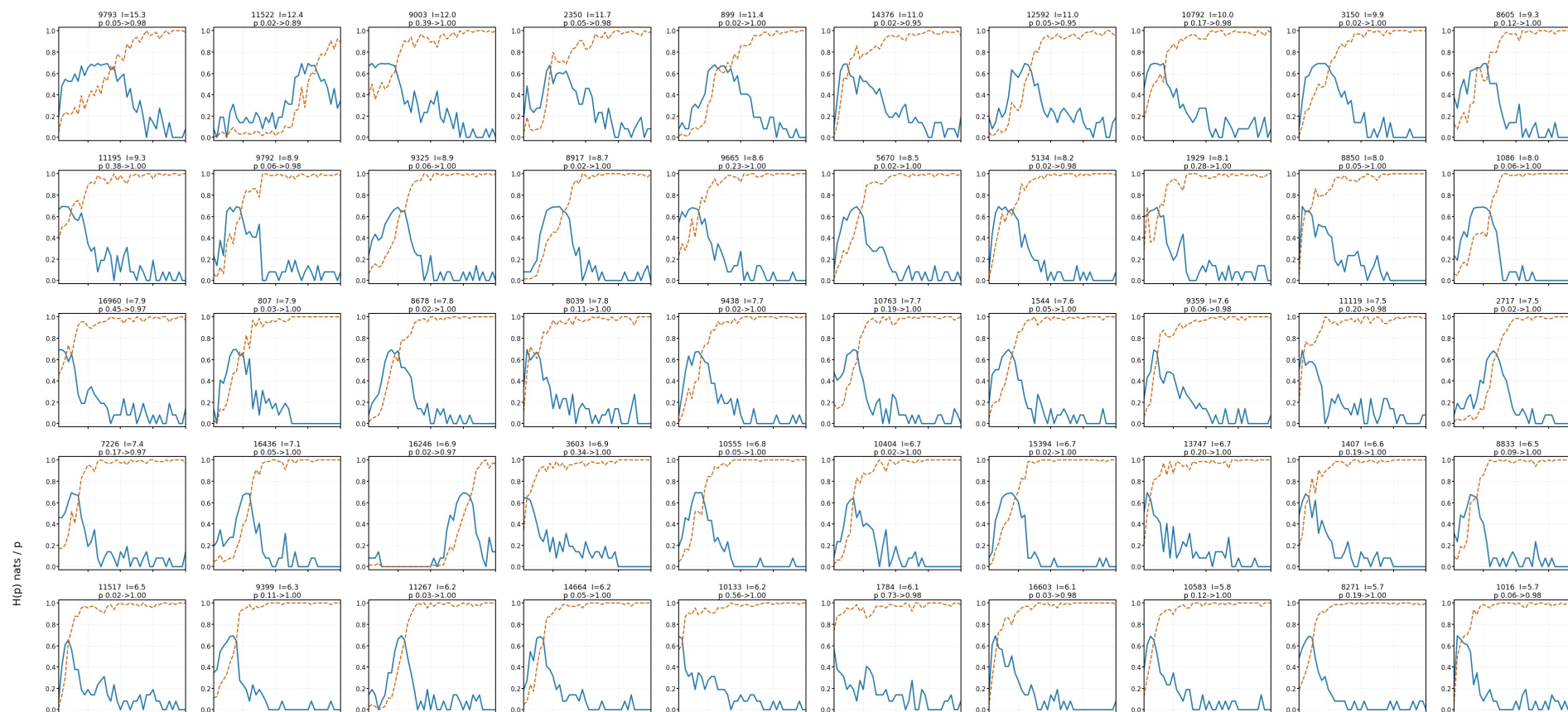


$$p(\text{win}) \approx 1/2, I_t = V[R_t] \approx 1/4$$

实验里 self-play 相比固定对手更稳，弱对手胜率也更高。

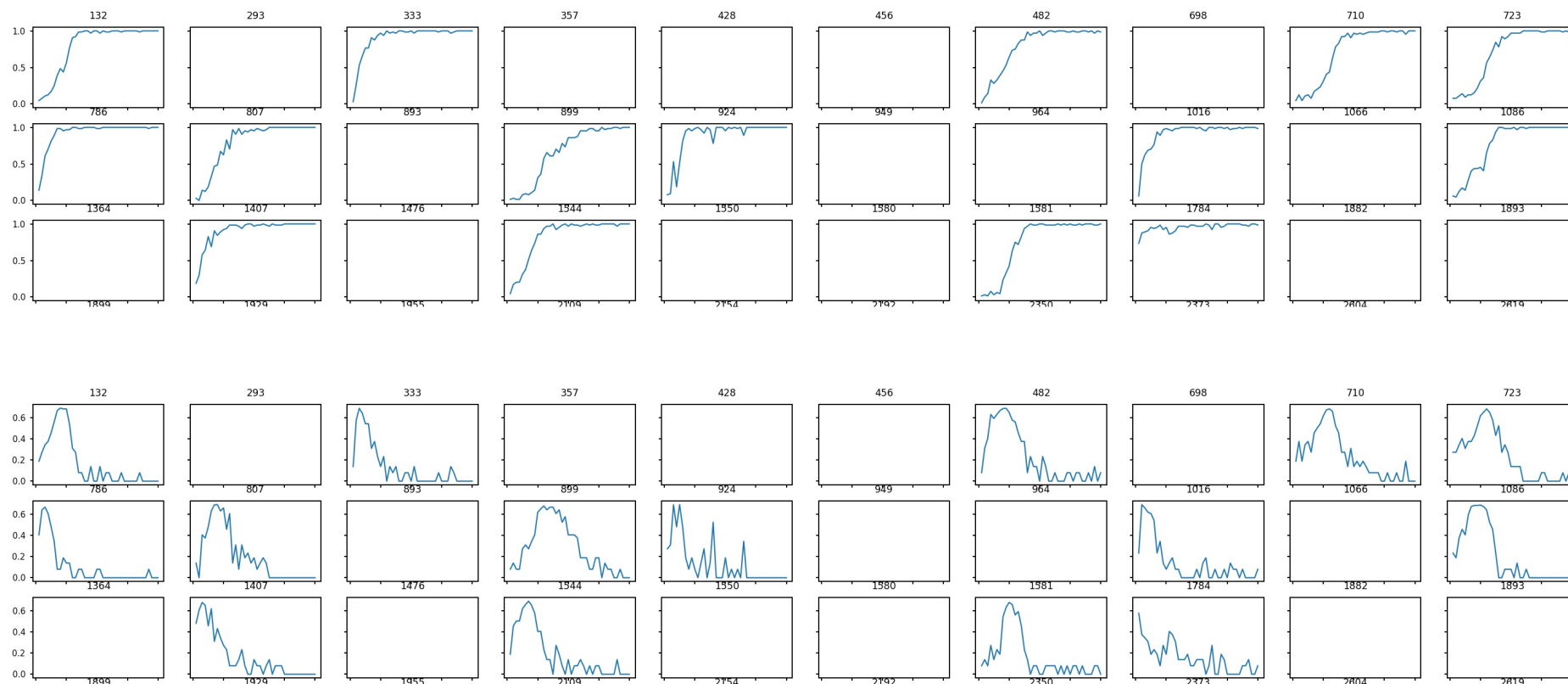


那么在数学题上呢？先看数据集只有 1 道题



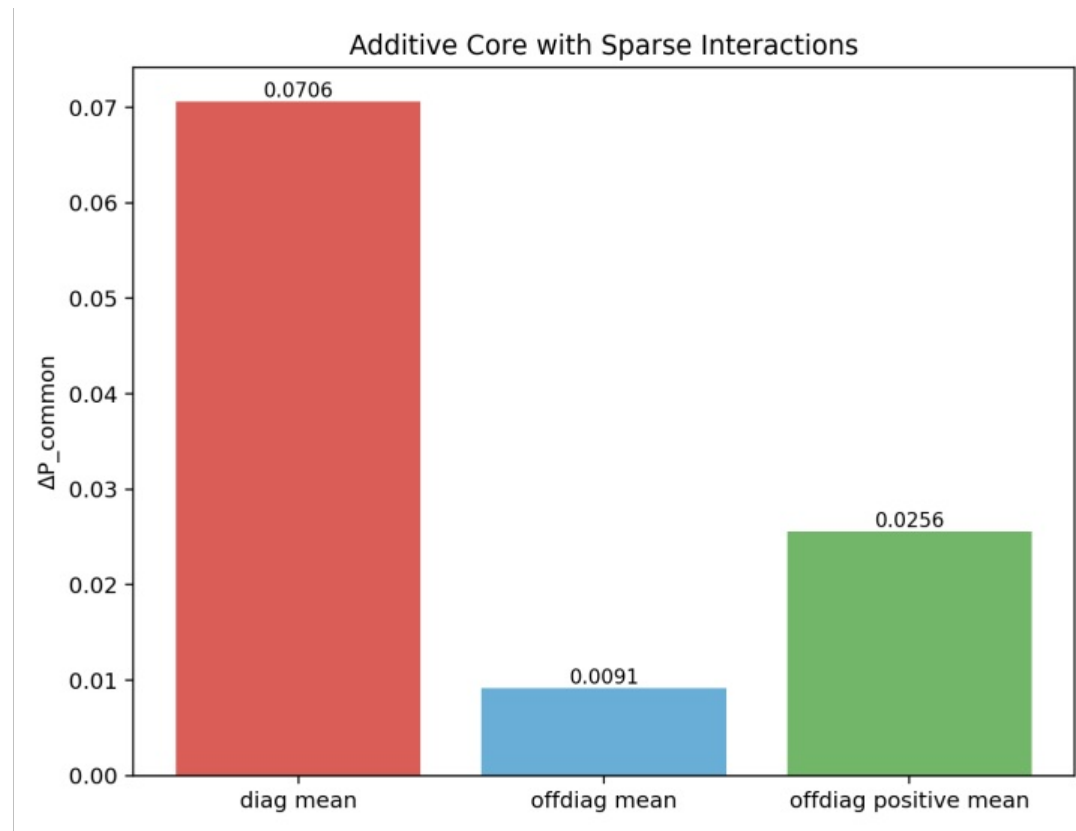
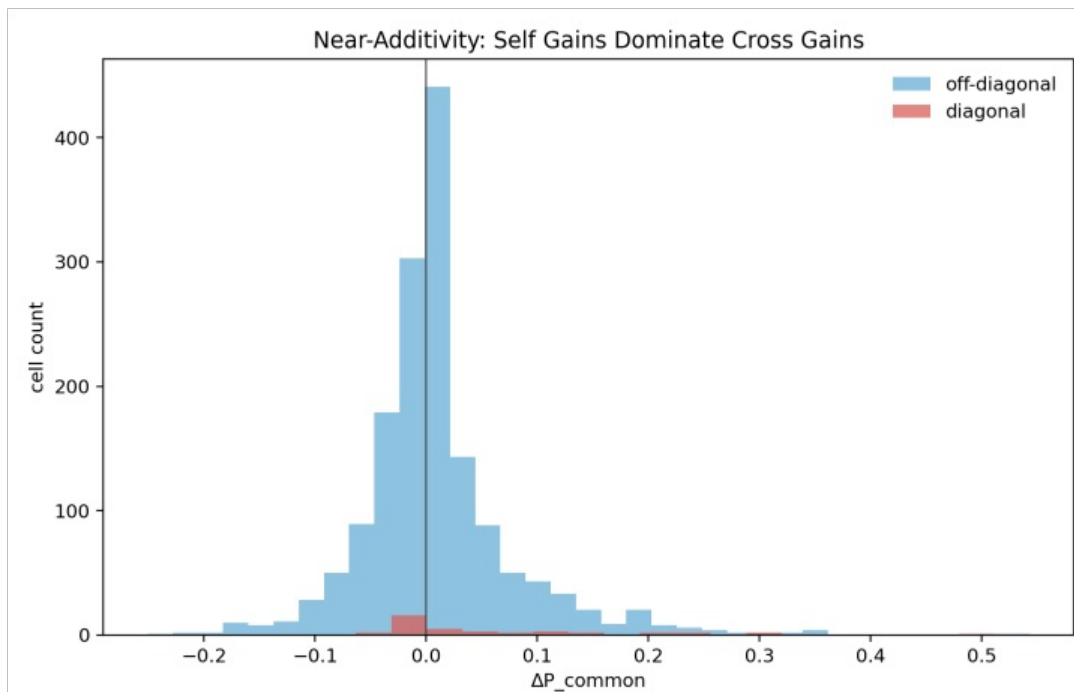
DAPO-17k completed single-question sweeps top-50: 每个小图都是“只训练这一道题”的独立实验；蓝线是 $H(p)$ ，橙虚线是 p 。 p 接近 1 后，信息量迅速回到 0。

再看数据集有多道题的情况



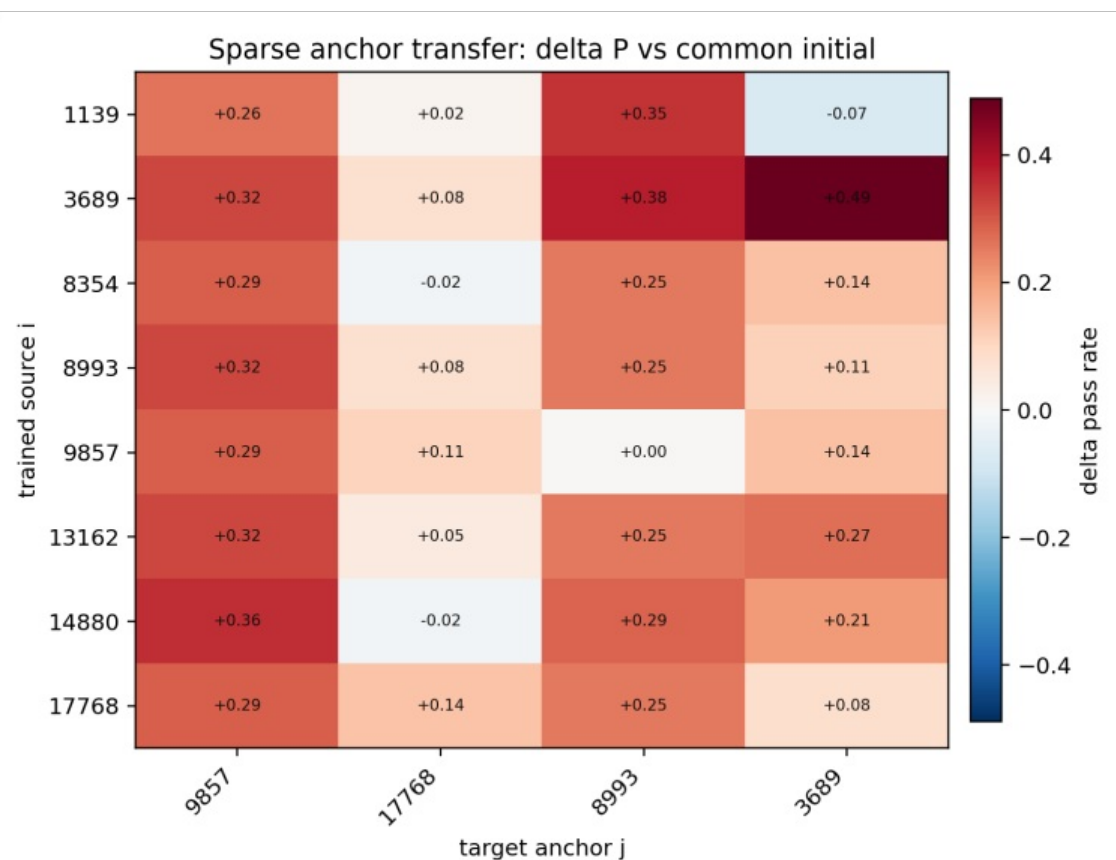
DAPO-17k sweep 前 30 个题目：上半部分是每题 pass-rate p 的轨迹，下半部分是同一批题的信息量 H / variance 轨迹。
多题数据集更像若干有限预算的叠加。

多道题：多数信号近似可加，但相互作用较弱



如果题目之间相互作用弱，那么增加题数可以增加总信息量；但总上限仍然随固定题集规模受限。

距离一：用性能向量



每一行就是一道题的 fingerprint

训练题目 i，然后在同一组 target / anchor 上测试性能。

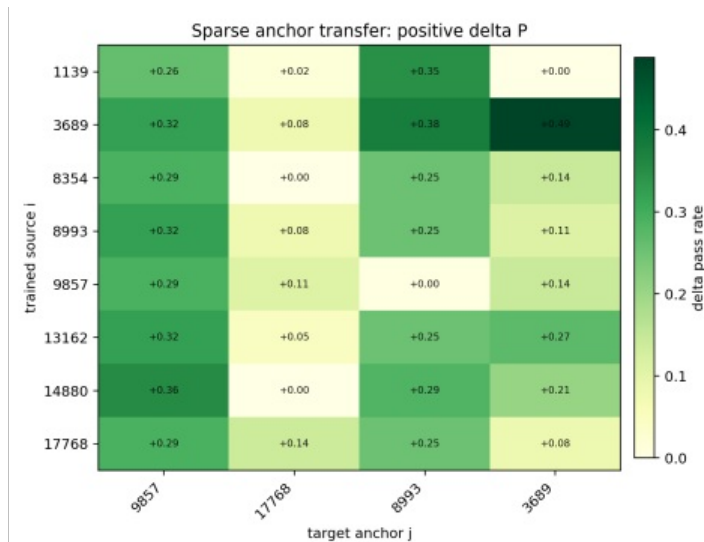
$$x_i = (p_{i1}, p_{i2}, \dots, p_{in})$$

如果两道题改善的是同一批 target，它们的方向相近。

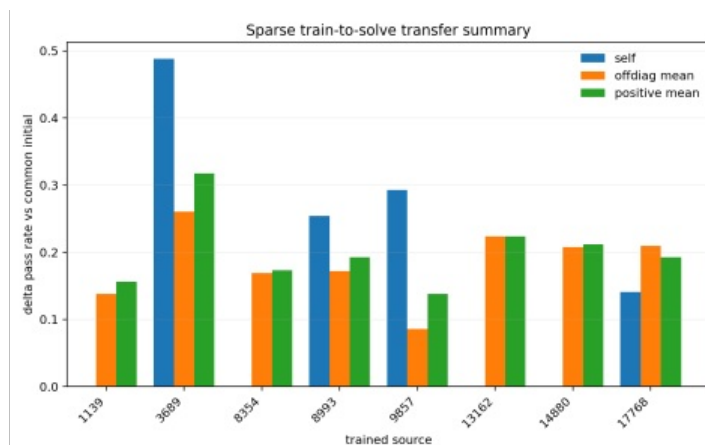
$$d_{\cos}(i, j) = 1 - \cos(x_i, x_j)$$

这给出一个近似无向的题目距离空间。

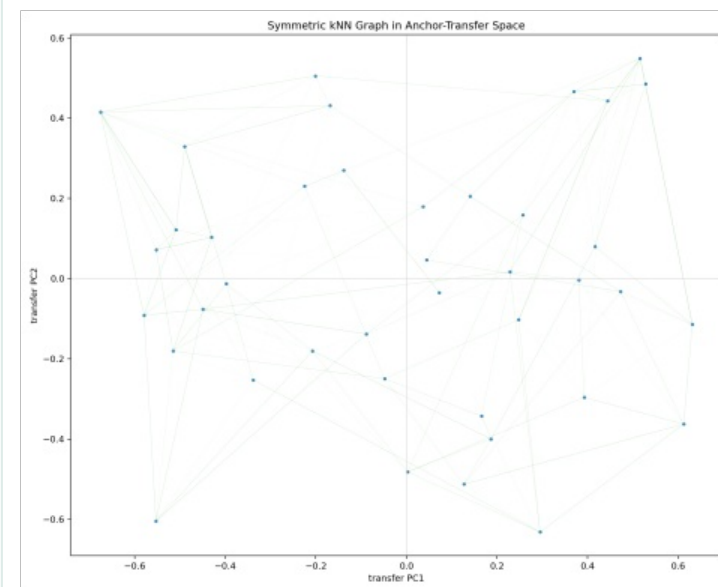
距离二：用自归一化迁移



p_{ij} : 训练 i 后, j 的性能提升。



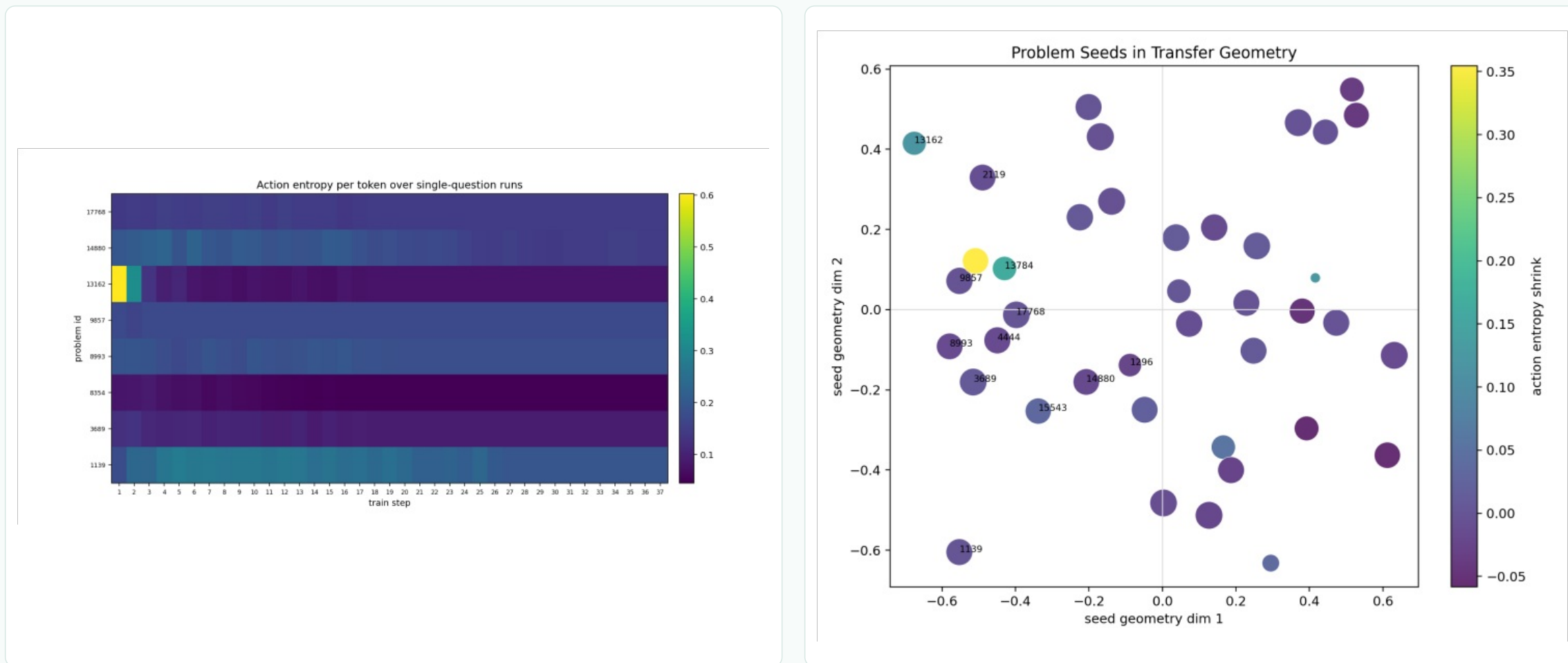
p_{ii} : 训练 i 后, i 自己的提升; 用它归一化不同 source。



R_{ij} 越大, i 对 j 的外溢越强; 保留大边即可画图。

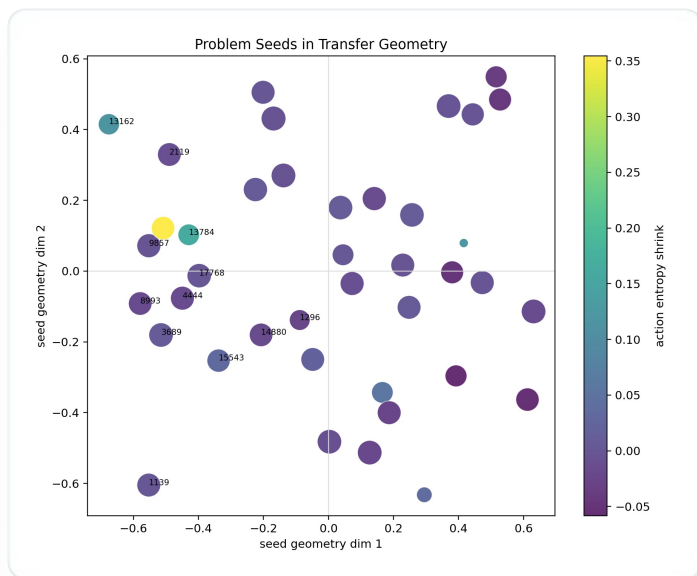
核心量很简单: $R_{ij} = p_{ij} / p_{ii}$ 。它天然是有方向的; 如果一定要对称距离, 可以再把 R_{ij} 和 R_{ji} 合并。

收缩和几何可以同时画出来



左边看收缩，右边看题目在 transfer space 中的位置。几何关系告诉我们哪些题可以互相带动。

固定数据集上的 RL：把西瓜缩成籽



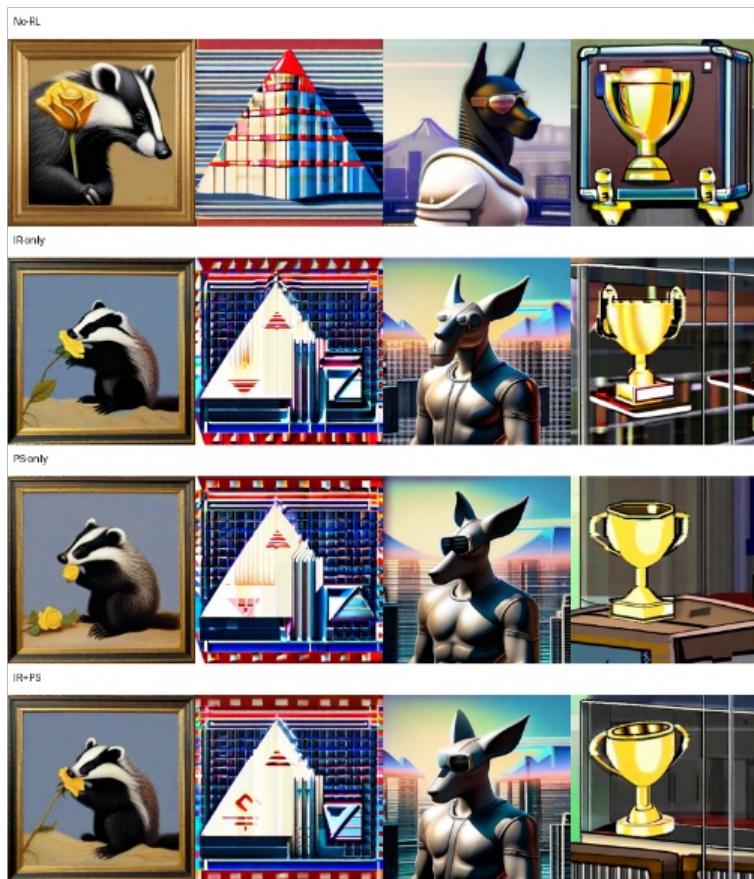
每个籽是一道题

原始模型是一个宽的输出分布。

固定题集上的 RL 会降低 action entropy，把概率质量收缩到若干个题目解法附近。

这些籽之间的距离，就是刚才由 transfer 定义出的数据集几何。

Diffusion RL：同一个信息瓶颈，也能在图 像里看到



四行分别是 No-RL、IR-only、PS-only、IR+PS；diffusion 版本把“题目”换成 prompt，把 rollout 换成生成图像/denoising trajectory。

SDXL + DDPO，10 个 prompt

每个 prompt 采样图像，再由 reward model 给出连续分数。

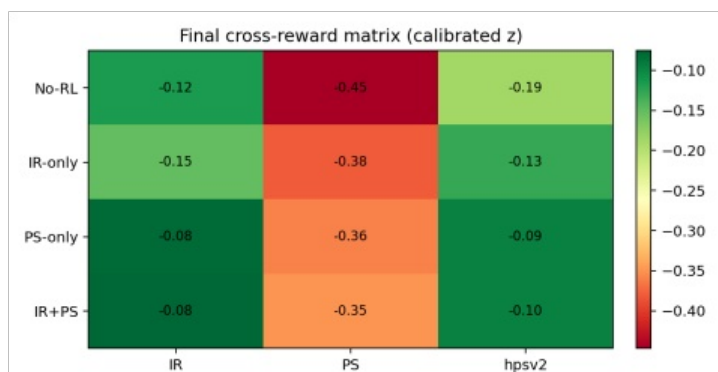
$$c_i \rightarrow x \sim \pi_t(\cdot | c_i) \rightarrow R(c_i, x) \in \mathbb{R}$$

这里不是 0/1 verifier，而是 ImageReward、PickScore、HPSv2 这样的 reward channel。

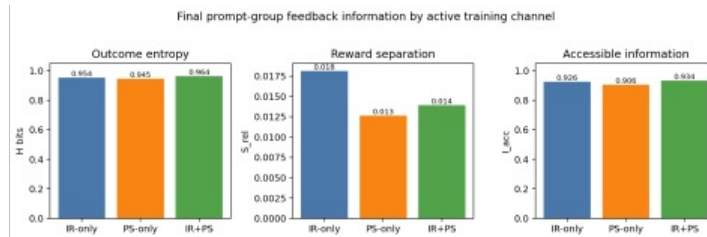
所以信息量看的是同一 prompt 下不同图像的 reward distribution 是否还有可区分宽度。

$$I_i(t) = \text{Var}_x[R(c_i, x)]$$

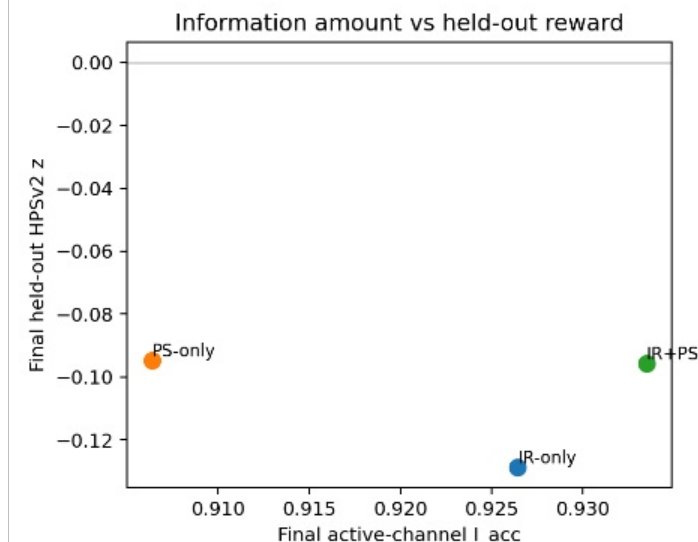
Diffusion 结果：多 reward 提供更宽的信息通道



最终 cross-reward matrix: IR+PS 在 ImageReward 和 PickScore 上最好, HPSv2 基本追平 PS-only。



最终 prompt-group 信息量: IR+PS 的 accessible information 最高。



信息量和 held-out HPSv2 的关系: IR+PS 有更丰富反馈, 同时接近最强单 reward。

结论要谨慎说: multi-reward 没有严格赢过最强 single reward, 但它给出最均衡的 reward profile, 并让 active feedback information 最高。

如果固定题集信息有限，怎么增加信息？

更多相对独立的题

如果题目基本独立，增加题数会近似增加总信息预算。

更丰富的 reward 信号

不仅看最终答案，还看 proof state、Lean 检查、步骤级 reward、partial credit。

生成式环境

Diffusion / theorem generation / AI task generation 可以不断制造新边界。

也就是说，未来真正需要的是会自我生成任务、验证任务、更新任务分布的系统。

最后回到一开始的问题

AlphaZero

对手会随着自己变强而变强。

新的棋局、新的边界、新的胜负差异，会被自博弈持续制造出来。

固定数学题集

题做不出来时，reward 几乎没有差异；题做出来后，reward 又很快失去差异。

RL 更像把一个宽分布压进少数稳定解法附近。

所以问题不是“RL 有没有用”，而是：信息源会不会继续长出来。